Neural Motion Blending Across Arbitrary Character Topologies

Luca Cazzola[1 [0009−0000−6285−8342]⋆], Giulia Martinelli[1,2 [0000−0003−3713−3053]⋆], and Nicola Conci[1,2 [0000−0002−7858−0928]]

[1] University of Trento, Trento 38123, Italy
luca.cazzola-1@studenti.unitn.it,
{giulia.martinelli-2,nicola.conci}@unitn.it
[2] CNIT, Trento 38123, Italy
https://mmlab-cv.github.io/neural_motion_blending/

Abstract. Motion blending in character animation enables the synthesis of new motions by interpolating between existing examples. Current methods are typically restricted to fixed skeleton topologies, requiring identical or near-identical skeletal structures across characters. We present a novel framework for motion blending across heterogeneous skeletons. The proposed architecture combines a semantic encoder, which extracts per-frame latent representations of the motion state, with a diffusion-based decoder, which reconstructs character-specific motion conditioned on this latent code. At inference, blended motions are obtained by interpolating the latent representations of two input motions. We train and evaluate the method on the Truebones Zoo dataset using motions defined on both same and distinct skeleton topologies, demonstrating the ability to achieve smooth and plausible blending in a variety of scenarios.

Keywords: Motion Blending · Diffusion Autoencoders · Any-Topology.

1 Introduction

Animation pipelines involve diverse casts of characters, from humanoids to creatures and stylized avatars, whose skeletal structures vary in topology, proportions, and bone hierarchy. Standard motion blending workflows are generally tied to a fixed skeleton representation, making cross-character interpolation hard to achieve. As a result, animators are often forced to rely on manual correspondence design, retargeting, and cleanup before they can test even simple blended motions [10]. Recent neural approaches have also begun to revisit motion blending as a learnable problem [11,21], aiming to provide more automatic and controllable tools for animators. Still, these approaches do not target cross-topology motion interpolation.

Yet, some progress has been made also in cross-topology motion synthesis [8,22] and retargeting [1,5,14], in part enabled by datasets offering heterogeneous skeletal structures [20]. However, in synthesis, motion is generated from

⋆ Denotes equal contribution.

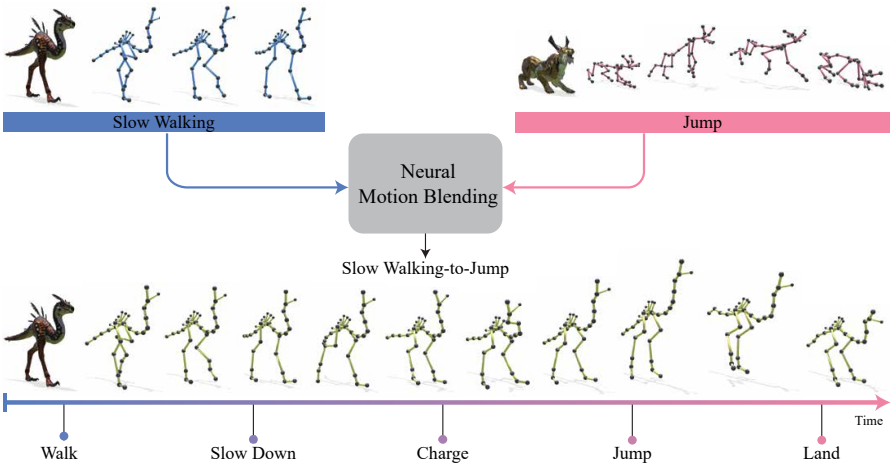

Fig. 1: We introduce a unified framework for cross-topology motion blending. Given reference motion and a target motion, our method generates a transition while preserving reference skeleton structure. Here, a slow-walking raptor is blended with a jumping lynx using the raptor as reference skeleton, producing a new raptor animation that walks, slows down, charges, jumps, and lands.

scratch, and with retargeting motion is transferred form one character to a second one; thus, neither of them directly addresses the problem of blending two existing motions, especially between different topologies as shown in Figure 1.

In this work, we introduce a skeleton-agnostic diffusion autoencoding framework for motion blending across heterogeneous characters. Our key idea is to decouple motion semantics from skeleton-specific realization by factorizing each input motion into a semantic code that captures the global motion state, and a stochastic code that captures fine-grained skeleton-specific variation conditioned on it. A graph-based semantic encoder maps motions from arbitrary topologies into a shared latent space, while a diffusion-based decoder reconstructs skeleton-specific motion conditioned on the semantic representation. Blending is then achieved by interpolating semantic codes and composing stochastic components at inference time. Although not the primary goal, the same representation also enables retargeting across heterogeneous skeletons, where we achieve state-of-the-art results against Motion2Motion [5] without requiring explicit joint-level correspondences.

Our contributions can be summarized as follows:

- We formalize the task of motion blending across arbitrary skeleton topologies, extending beyond settings that assume identical skeletons.
- We introduce a skeleton-agnostic diffusion autoencoding framework that factorizes motion into semantic and stochastic latent components, enabling cross-topology blending.

– We demonstrate generalization across heterogeneous skeletons, achieving state-of-the-art performance on the Truebones Zoo dataset [20], without requiring explicit joint-level correspondences.

2 Related Work

Motion blending. Motion blending is a core technique in character animation, enabling the synthesis of new motions by interpolating between existing clips in a continuous control space. Classical approaches addressed this problem through motion graphs and statistical formulations of motion interpolation, balancing smoothness, controllability, and efficiency [7,12,15]. More recent work has revisited related problems with generative models, including text-conditioned action composition [2], motion editing through blending-based augmentation [11], seamless transition generation [3], diffusion-based motion priors for transition synthesis [18]. Most directly related to blending as a dedicated task, [21] introduced a single-shot framework for controllable animation blending through temporal conditioning. Despite their differences, these methods operate on fixed-dimensional motion representations tied to a predefined joint set, and therefore assume a shared skeleton topology. By contrast, our work addresses motion blending across characters with heterogeneous skeletal structures, a setting that, to the best of our knowledge, has not been explicitly studied in prior work.

Cross-topology motion synthesis and retargeting. Related problems have received increasing attention in motion retargeting and synthesis. Retargeting aims to transfer a motion from a source character to a target character with a different skeletal structure while preserving kinematic plausibility despite differences in hierarchy, proportions, and number of joints. Aberman *et al.* [1] introduced one of the first retargeting methods for heterogeneous skeletons through skeleton-aware convolutions and differentiable pooling. MoMa [14] extended this setting with a masked pose autoencoder that supports isomorphic, homeomorphic, and non-homeomorphic skeletons, including characters with additional kinematic chains such as tails. However, these methods are primarily developed and evaluated on humanoid characters, rather than broader cross-species topology variations. More recently, Motion2Motion [5] addressed cross-topology motion transfer by formulating retargeting as a conditional patch-based motion matching problem under sparse joint correspondences. Unlike standard retargeting methods, which transfer a source motion to a target skeleton in rest pose, Motion2Motion requires both source and target motion sequences and relies on temporal matching between them. This formulation broadens retargeting beyond the standard transfer setting, but also introduces a dependence on motion-level alignment and sparse structural correspondences between the two skeletons. Recent motion synthesis methods have instead moved toward topology-agnostic generation. AnyTop [8] studies motion synthesis conditioned on arbitrary skeletal structures, enabling generation across heterogeneous rigs rather than transfer between matched characters. Despite this progress, it focuses on generating motion for individual skeletons rather than blending between motion sources.

3 Method

3.1 Preliminaries

Diffusion Autoencoders. Diffusion Autoencoders (DiffAE) [17] augment diffusion models with an explicit semantic representation. Given a clean sample x_0, a semantic encoder extracts a code $z_{\text{sem}} = E_\phi(x_0)$, while the residual stochastic component is represented by x_T, obtained through DDIM inversion [19]. Each sample is thus represented by the pair $z = (z_{\text{sem}}, x_T)$, where z_{sem} captures high-level semantic content and x_T preserves fine-grained detail. We adopt the same factorized view in the motion domain, with the additional requirement that the semantic representation be shared across heterogeneous skeletal topologies.

Motion Diffusion on Arbitrary Skeletons. AnyTop [8] is a diffusion model for motion generation across arbitrary skeletal topologies. A skeleton is represented as $\mathcal{S} = \{\mathcal{P}_\mathcal{S}, \mathcal{R}_\mathcal{S}, \mathcal{D}_\mathcal{S}, \mathcal{N}_\mathcal{S}\}$, where $\mathcal{P}_\mathcal{S}$ denotes the rest pose, $\mathcal{R}_\mathcal{S}$ the pairwise joint relations, $\mathcal{D}_\mathcal{S}$ the topological distances, and $\mathcal{N}_\mathcal{S}$ the joint names. A motion sequence is represented as $X \in \mathbb{R}^{N \times J \times D}$, where N is the number of frames, J the number of joints, and $D = 13$ the feature dimension per joint, including root-relative position, 6D rotation [23], linear velocity, and foot-contact label. We use AnyTop as the diffusion backbone for motion decoding.

3.2 Architecture

Fig. 2 provides an overview of the proposed architecture, which consists of a *Semantic Encoder* and a *Stochastic Decoder*. We adopt the motion representation introduced in Sec. 3.1 and build on the AnyTop backbone [8]. In particular, we use the same `Enrichment Block` to embed the input motion together with skeleton-specific priors, including the rest pose $\mathcal{P}_\mathcal{S}$ and the joint descriptions $\mathcal{N}_\mathcal{S}$, and the same Spatio-Temporal Transformer `STT backbone`, whose layers combine topology-aware spatial attention with local temporal attention through $\mathcal{R}_\mathcal{S}$ and $\mathcal{D}_\mathcal{S}$. The same `Enrichment Block` and `STT backbone` are used for both the clean motion X_0 in the *Semantic Encoder* and the noisy motion X_t in the *Stochastic Decoder*. We refer the reader to [8] for further details.

Semantic Encoder. Inspired by Diffusion Autoencoders [17], we introduce a *Semantic Encoder* to extract a compact semantic representation from the clean motion and use it to condition the stochastic generation process. The encoder processes X_0 producing a per-frame latent representation $z_{\text{sem}} \in \mathbb{R}^{N \times d}$, where d is the embedding dimension of the `STT backbone`. Each latent vector summarizes the global semantic state of the motion at the corresponding frame, including high-level aspects such as motion category, phase, and overall pose dynamics. To enforce a fixed-dimensional representation independently of skeleton topology or joint count, we apply Global Average Pooling (GAP) across the joint dimension at the output of the feed-forward block, yielding a simple topology-agnostic bottleneck.

Stochastic Decoder. Given a noisy motion X_t at timestep t, the decoder predicts the corresponding clean motion $\hat{X}_0$ conditioned on the semantic latent

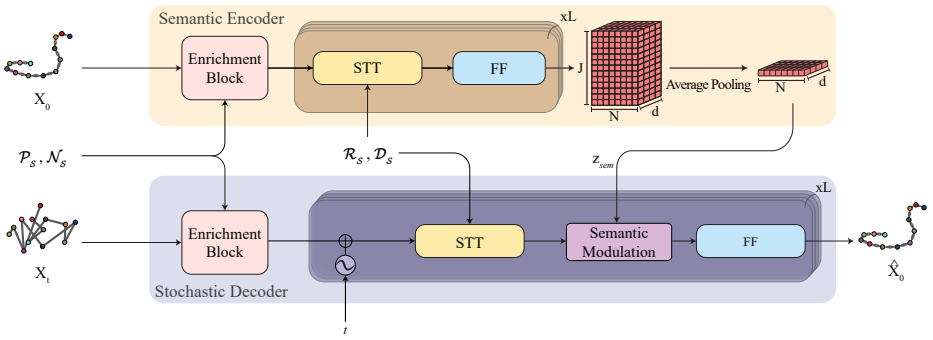

Fig. 2: **The proposed architecture.** Our framework consists of a *Semantic Encoder* and a *Stochastic Decoder*. The former processes the clean motion X_0 through the `Enrichment Block` and `STT backbone`, then applies average pooling over joints to obtain the semantic code z_{sem}. The latter takes noisy motion X_t and predicts clean motion $\hat{X}_0$, conditioned through the `Semantic Modulation`.

z_{sem}. As in the *Semantic Encoder*, the input is first processed and conditioning is enabled by the `Semantic Modulation` module inserted after each STT layer. Rather than using standard cross-attention, we adopt a FiLM-style conditioning strategy better matched to our setting, where conditioning is provided by a single vector per frame. With a single key-value token, the attention distribution becomes trivial, and the output reduces to an additive projection of the semantic code without meaningful token-wise interaction. Inspired by FiLM [16] and Gated Linear Units [6], we instead use a learned modulation mechanism that provides joint-wise multiplicative conditioning. Formally, given a joint feature $h_{n,j}$ and the semantic code $z_{\text{sem}}^{(n)}$ at frame n, the modulation is defined as

$$\tilde{h}_{n,j} = h_{n,j} + g(h_{n,j}, z_{\text{sem}}^{(n)}) \odot \phi(z_{\text{sem}}^{(n)}), \tag{1}$$

where $g(\cdot)$ is a learned sigmoid gating function implemented as a two-layer MLP, $\phi(\cdot)$ projects the semantic code to the joint feature space, and $\odot$ denotes element-wise multiplication. Since $g(\cdot)$ depends through concatenation on both $h_{n,j}$ and $z_{\text{sem}}^{(n)}$, each joint is free to modulate semantic information according to its local feature state. This conditioning mechanism allows the decoder to remain controllable when driven by interpolated semantic codes at inference time.

3.3 Training Objective.

We adopt the same training objective as AnyTop [8]. Given a clean motion X_0, its noised counterpart X_t, and the target skeleton $\mathcal{S}$, the model predicts the clean sample $\hat{X}_0$. Training is based on a reconstruction loss on $\hat{X}_0$, complemented by a geodesic loss on joint rotations to better reflect distances on the rotation manifold. The final objective is

$$\mathcal{L} = \mathcal{L}_{\text{simple}} + \lambda_{\text{rot}} \mathcal{L}_{\text{rot}}, \tag{2}$$

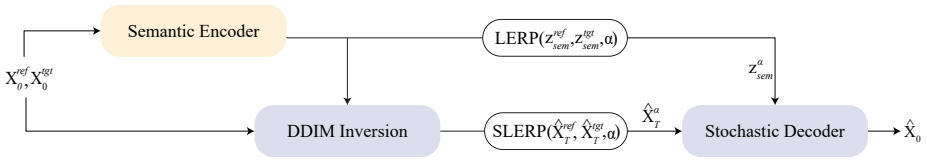

Fig. 3: **Inference: Sampling and Blending.** The reference and target motions (X_0^{ref}, X_0^{tgt}) are encoded into semantic latents and inverted stochastic subcodes. Semantic blending is performed with LERP, while stochastic blending is performed with SLERP. The resulting codes, z^{α}_{sem} and $\hat{X}^{\alpha}_T$, are finally decoded into the blended motion $\hat{X}_0$.

where $\mathcal{L}_{\text{simple}}$ is an ℓ_2 loss on the predicted clean motion and $\mathcal{L}_{\text{rot}}$ is computed on rotation matrices obtained from the 6D representation via Gram–Schmidt [23]. The *Semantic Encoder* is trained jointly with the *Stochastic Decoder* through the same objective. Following DiffAE [17], no explicit regularization is applied to z_{sem}; instead, the information bottleneck induced by GAP provides the structural constraint that organizes the latent space, as supported by the interpolation results discussed in Sec. 4.

3.4 Inference: Sampling and Blending

At inference, our method generates motion blends by manipulating the factorized latent code $z = (z_{\text{sem}}, \hat{X}_T)$, where $\hat{X}_T$ denotes the stochastic subcode recovered by DDIM inversion. Fig. 3 illustrates the full pipeline. Given two input motions, a reference motion X_0^{ref} and a target motion X_0^{tgt}, the goal is to synthesize an output motion $\hat{X}_0$ on the reference skeleton $\mathcal{S}$ while transitioning from the reference to the target according to a temporal blending schedule $\alpha(n) \in [0, 1]$, where n indexes the output frames. For readability, in the following we write α in place of the frame-dependent blending coefficient $\alpha(n)$. In the semantic branch, the clean motions are independently encoded by the *Semantic Encoder*, producing semantic latents $z_{\text{sem}}^{\text{ref}}$ and $z_{\text{sem}}^{\text{tgt}}$. In the stochastic branch, both motions undergo DDIM inversion [19], yielding stochastic subcodes $\hat{X}_T^{\text{ref}}$ and $\hat{X}_T^{\text{tgt}}$. Following [17], the semantic and stochastic subcodes are blended through

$$z^{\alpha}_{\text{sem}} = \text{LERP}\big(z^{\text{ref}}_{\text{sem}}, z^{\text{tgt}}_{\text{sem}}, \alpha\big)\,, \qquad \hat{X}^{\alpha}_T = \text{SLERP}\Big(\hat{X}^{\text{ref}}_T, \hat{X}^{\text{tgt}}_T, \alpha\Big)\,. \tag{3}$$

Applying SLERP to the stochastic subcodes preserves the norm of the interpolated noise, which is expected to lie on the Gaussian shell induced by the diffusion process. In the in-skeleton case, both the semantic and stochastic subcodes can be directly interpolated. In the cross-skeleton case instead $\hat{X}_T^{\text{tgt}}$ cannot be meaningfully reused, as stochastic subcodes are intrinsically tied to their relative skeleton topology. We therefore replace $\hat{X}_T^{\text{tgt}}$ with unit Gaussian noise $\epsilon \in \mathcal{N}(0, I)$ while keeping the semantic interpolation unchanged. This preserves the semantic transition, yet allowing the decoder to synthesize topology-consistent structural

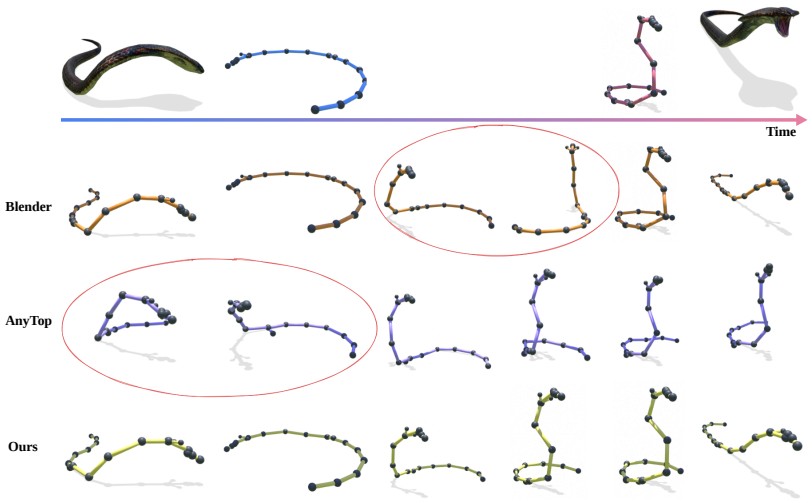

Fig. 4: **Qualitative results: (A) In-Skeleton Blending.**

Table 1: (A) In-Skeleton Blending.

Method	FID ↓	Proximity ↓	Jerk Ratio ∼ 1	IntraDivDiff ↓
Blender NLA [4]	15.14 ± 0.60	0.15 ± 0.15	**1.64 ± 1.31**	0.11 ± 0.14
AnyTop (DDIM) [8]	19.24 ± 0.45	0.20 ± 0.12	5.45 ± 10.09	0.13 ± 0.09
Ours (DDIM)	**13.99 ± 0.32**	**0.13 ± 0.08**	1.83 ± 1.57	**0.10 ± 0.06**

details for the output skeleton $\mathcal{S}$, at the cost of losing fine-grained stochastic details from the target motion.

The blended semantic code and the corresponding stochastic subcode are finally passed to the *Stochastic Decoder*, which synthesizes the output motion $\hat{X}_0$ on $\mathcal{S}$ skeleton. When at least one stochastic subcode matches $\mathcal{S}$ structure, the model relies on DDIM sampling to exploit stochastic subcodes, otherwise, as in cross-skeleton transfer, the model applies Denoising Diffusion Probabilistic Model (DDPM) sampling [9]. In this case, the stochastic component is randomly sampled rather obtained through DDIM inversion.

4 Experiments

4.1 Experimental Setup

Datasets. We evaluate our method on the Truebones Zoo dataset [20]. Truebones contains 1,219 motion clips (147,178 frames) spanning 70 heterogeneous skeletons, including mammals, birds, insects, dinosaurs, fish, and snakes. Skeletons vary substantially in joint count, connectivity, root definition, scale, and naming convention, making the dataset particularly well suited for evaluating cross-topology motion blending.

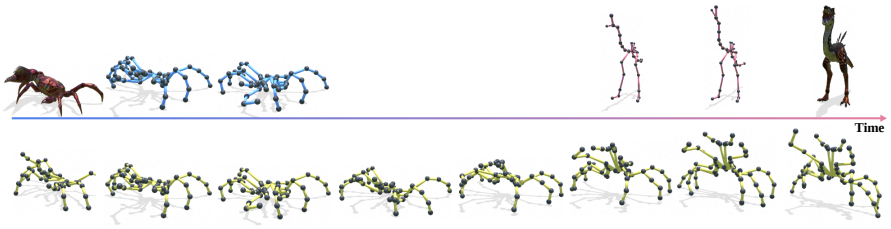

Fig. 5: **Qualitative results: (B) Cross-Skeleton Blending.**

Table 2: (B) Cross-Skeleton Blending.

Method	FID ↓	Proximity ↓	Jerk Ratio ∼ 1	IntraDivDiff ↓
Ours (DDPM)	$\underline{17.80 \pm 0.38}$	$\underline{0.22 \pm 0.13}$	$\underline{3.52 \pm 5.63}$	$\underline{0.11 \pm 0.08}$
Ours (DDIM)	$\mathbf{16.81 \pm 0.35}$	$\mathbf{0.20 \pm 0.12}$	$\mathbf{3.51 \pm 5.64}$	$\mathbf{0.10 \pm 0.08}$

Evaluation Metrics. All metrics are computed over the transition region. Proximity [13] and FID [21] assess coherence with the input motions. Proximity operates in the joints space and is therefore restricted to the in-skeleton setting, where comparison against the charachter's real motion pool (GT pool) is well posed. FID is computed in the latent space of the semantic encoder, as no pretrained motion feature extractor generalizes across the heterogeneous topologies considered in our evaluation; to mitigate potential bias, we complement it with joint-space metrics through root relative positions whenever they provide meaningful comparison. *IntraDivDiff* [8] measures whether the internal variability of the transition matches the character's average motion statistics, while *Jerk Ratio* measures smoothness as the ratio between the generated jerk and the mean jerk of the corresponding GT pool, with values close to 1 indicating more common dynamics. In the retargeting setting we report only *FID* and *Jerk Ratio*.

Baselines. We compare against both classical and learning-based baselines. **Blender** [4] performs geometry-level blending directly in pose space through the Blender Nonlinear Animation editor, without any learned component, and serves as the standard production-level reference. **AnyTop** [8] is used as a learning-based baseline. We adapt it to the blending setting by applying DDIM inversion to both source motions and interpolating the resulting stochastic codes with SLERP over the transition region. For the retargeting setting on Truebones, we additionally include **M2M** [5] as a baseline for cross-character motion transfer.

Setup. We evaluate the proposed framework in two main settings: **(A) In-Skeleton Blending**, where both input motions share the same skeletal topology $\mathcal{S}$, and **(B) Cross-Skeleton Blending**, where the two motions originate from characters with different topologies. Unless otherwise specified, we use an *ease-in-ease-out* blending schedule with strength 1.0 for $\alpha(n)$; the effect of this choice is analyzed in Sec. 4.3. We further consider the special case $\alpha(n) = 1$, corresponding to **(C) Motion Retargeting** across different skeletons. Training is

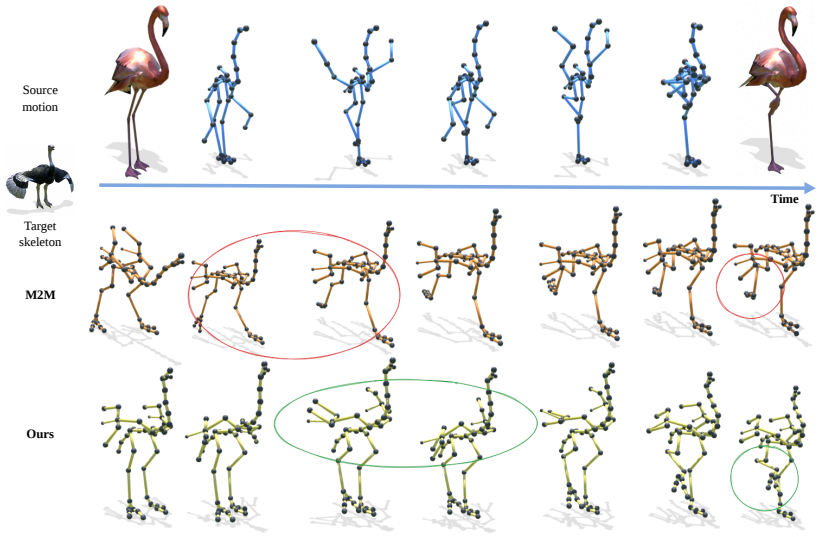

Fig. 6: **Qualitative results: (C) Retargeting.**

performed on the entire Truebones dataset, which lacks inherent ground truths for blending or retargeting tasks. For validation, we randomly pair 5 reference motions per character with 5 target motions. We select motions with at least 40 frames and set a blending region of 30 frames, corresponding to 1.5 seconds.

4.2 Results

In Tabs. 1 to 4 the **best** and second best scores per metric are respectively shown in bold and underlined. Qualitative video results are provided in the *Supp. Mat.*
(A) In-Skeleton Blending. The goal is to generate a blended motion on skeleton $\mathcal{S}$ from two input motions performed by the same character. Tab. 1 reports the quantitative results. Our model outperforms all baselines in terms of *FID*, *Proximity*, and *IntraDivDiff*, while maintaining a competitive *Jerk Ratio*. AnyTop yields the weakest performance, while Blender produces smoother motions at the cost of poorer transition quality. This trend is visible in Fig. 4: AnyTop drifts from the reference already at the beginning of the transition, whereas Blender fails to reproduce the correct global reorientation. By contrast, our method produces a gradual and coherent transition that remains faithful.
(B) Cross-Skeleton Blending. In this setting, the two input motions are performed by different characters with different skeletal topologies. Since a direct comparison in the joint space is not applicable across heterogeneous skeletons, and neither Blender NLA nor AnyTop supports cross-skeleton blending, we report results only for our method, using both standard DDIM sampling and stochastic DDPM sampling. The two variants yield comparable results, with

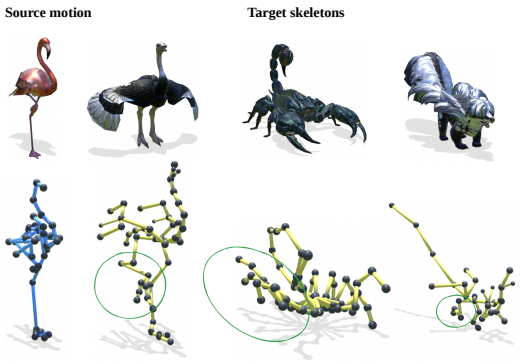

Fig. 7: Retargeting to many.

Table 3: (C) Retargeting.

Method	FID ↓	Jerk Ratio ~ 1
M2M [5]	$\underline{31.31 \pm 0.72}$	$\mathbf{1.64 \pm 4.11}$
Ours (DDPM)	$\mathbf{9.64 \pm 0.32}$	$\underline{2.40 \pm 3.42}$

DDIM performing slightly better overall, while DDPM highlights the stochastic potential of the framework by generating multiple plausible alternatives from the same input pair. Qualitative results are shown in Fig. 5, with additional video examples provided in the *Supp. Mat.* The generated motion remains faithful to the reference at the beginning of the sequence, then progressively shifts toward the target behavior while respecting the reference topology.

(C) Retargeting. When setting constant $\alpha(n) = 1$ our framework reduces to motion retargeting. Since the Truebones dataset does not provide paired motions, we evaluate this setting using only *FID* and *Jerk Ratio*. We additionally compare against M2M [5], which is the only existing baseline in our comparison specifically designed for retargeting. Since M2M relies on sparse correspondences and motion matching, Table 3 compares only walking motions with manually aligned leg correspondences. Tab. 3 shows that our model outperforms M2M by a significant margin in *FID*, while maintaining a competitive *Jerk Ratio*. M2M relies on sparse joint correspondences and semantically similar motion pairs. Our method achieves stronger results without requiring either. Moreover, although *FID* is computed in the latent space of our encoder for both methods, the qualitative comparison in Fig. 6 is consistent with the quantitative results: M2M fails to reproduce target semantics faithfully, whereas our output remains both faithful to target motion and structurally plausible on reference topology. Additional qualitative results across different characters are presented in Fig. 7, demonstrating consistent retargeting across topologies with varying joint counts and limb structures.

4.3 Ablation Studies

Tab. 4 reports two ablations in the in-skeleton setting. First, removing rotation (*w/o Rot*) or velocity (*w/o Vel*) features degrades performance on nearly all metrics, showing that both cues are important for stable and coherent blending. Second, we compare different blending schedules $\alpha(n)$, namely Linear interpolation and Ease interpolation with two strengths. The results are largely

Table 4: Ablations Studies.

Ablation	Setting	FID ↓	Proximity ↓	Jerk Ratio ∼ 1	IntraDivDiff ↓
In-Feats	w/o Rot	10.72 ± 0.65	0.13 ± 0.06	2.59 ± 4.34	$\mathbf{0.11 \pm 0.07}$
	w/o Vel	10.63 ± 0.66	0.13 ± 0.07	1.81 ± 1.28	$\mathbf{0.11 \pm 0.07}$
	Full	$\mathbf{9.75 \pm 0.62}$	$\mathbf{0.12 \pm 0.06}$	$\mathbf{1.65 \pm 0.88}$	$\mathbf{0.11 \pm 0.07}$
$\alpha(n)$	Linear	9.76 ± 0.60	$\mathbf{0.12 \pm 0.06}$	$\mathbf{1.48 \pm 0.71}$	0.11 ± 0.07
	Ours (Ease 1.0)	$\mathbf{9.75 \pm 0.62}$	$\mathbf{0.12 \pm 0.06}$	1.65 ± 0.88	0.11 ± 0.07
	Ease 2.0	9.78 ± 0.63	$\mathbf{0.12 \pm 0.06}$	1.80 ± 1.01	$\mathbf{0.10 \pm 0.07}$

comparable, with the main variation appearing in *Jerk Ratio*: stronger easing introduces larger velocity changes and therefore slightly less smooth transitions. We use Ease with strength 1.0 in the final model, as it provides the best trade-off between preserving motion dynamics and maintaining smooth transitions.

5 Conclusions

We presented a unified framework for motion blending across arbitrary skeletal topologies. By factorizing motion into a shared semantic code and a skeleton-specific stochastic component, our method enables blending between motions performed by different characters without requiring joint correspondences. Experiments on the Truebones Zoo dataset show remarkable performance in both in-skeleton and cross-skeleton settings, while the same representation also supports motion retargeting. We believe this work opens to a new direction for motion authoring tools that are less constrained by fixed skeleton representations and better aligned with the diversity of production animation pipelines.

Limitations. In the cross-skeleton and retargeting settings, the target stochastic code is replaced with Gaussian noise, which discards part of available target motion information. Additionally, transitions between semantically distant motions may exhibit artifacts, as the interpolated semantic trajectory must cover a large latent distance without explicit guidance on intermediate poses. Incorporating transition planning or intermediate pose conditioning could help resolve these cases and improve overall blending quality.

References

1. Aberman, K., Li, P., Lischinski, D., Sorkine-Hornung, O., Cohen-Or, D., Chen, B.: Skeleton-aware networks for deep motion retargeting. TOG (2020). https://doi.org/10.1145/3386569.3392462
2. Athanasiou, N., Petrovich, M., Black, M.J., Varol, G.: Teach: Temporal action composition for 3d humans. 3DV (2022). https://doi.org/10.1109/3DV57658.2022.00053
3. Barquero, G., Escalera, S., Palmero, C.: Seamless human motion composition with blended positional encodings. In: CVPR (2024). https://doi.org/10.1109/CVPR52733.2024.00051

4. Blender: Blender - the free and open source 3d creation software. https://www.blender.org/, last accessed: June 2026
5. Chen, L.H., Zhang, Y., Yin, Z., Dou, Z., Chen, X., Wang, J., Komura, T., Zhang, L.: Motion2motion: Cross-topology motion transfer with sparse correspondence. In: SIGGRAPH Asia (2025). https://doi.org/10.1145/3757377.3763811
6. Dauphin, Y.N., Fan, A., Auli, M., Grangier, D.: Language modeling with gated convolutional networks. In: ICML (2017)
7. Feng, A.W., Huang, Y., Kallmann, M., Shapiro, A.: An analysis of motion blending techniques. In: MIG (2012). https://doi.org/10.1007/978-3-642-34710-8_22
8. Gat, I., Raab, S., Tevet, G., Reshef, Y., Bermano, A.H., Cohen-Or, D.: Anytop: Character animation diffusion with any topology. In: SIGGRAPH (2025). https://doi.org/10.1145/3721238.3730621
9. Ho, J., Jain, A., Abbeel, P.: Denoising diffusion probabilistic models. In: NeurIPS (2020)
10. Hsieh, M.K., Chen, B.Y., Ouhyoung, M.: Motion retargeting and transition in different articulated figures. In: CAD/Graphics (2005). https://doi.org/10.1109/CAD-CG.2005.59
11. Jiang, N., Li, H., Yuan, Z., He, Z., Chen, Y., Liu, T., Zhu, Y., Huang, S.: Dynamic motion blending for versatile motion editing. In: CVPR (2025). https://doi.org/10.1109/CVPR52734.2025.02117
12. Kovar, L., Gleicher, M., Pighin, F.: Motion graphs. TOG (2002). https://doi.org/10.1145/566654.566605
13. Li, P., Aberman, K., Zhang, Z., Hanocka, R., Sorkine-Hornung, O.: Ganimator: Neural motion synthesis from a single sequence. TOG (2022). https://doi.org/10.1145/3528223.3530157
14. Martinelli, G., Garau, N., Bisagno, N., Conci, N.: Moma: Skinned motion retargeting using masked pose modeling. CVIU (2024). https://doi.org/10.1016/J.CVIU.2024.104141
15. Mukai, T., Kuriyama, S.: Geostatistical motion interpolation. TOG (2005). https://doi.org/10.1145/1073204.1073313
16. Perez, E., Strub, F., De Vries, H., Dumoulin, V., Courville, A.: FiLM: Visual reasoning with a general conditioning layer. In: AAAI (2018). https://doi.org/10.1609/AAAI.V32I1.11671
17. Preechakul, K., Chatthee, N., Wizadwongsa, S., Suwajanakorn, S.: Diffusion autoencoders: Toward a meaningful and decodable representation. In: CVPR (2022). https://doi.org/10.1109/CVPR52688.2022.01036
18. Shafir, Y., Tevet, G., Kapon, R., Bermano, A.H.: Human motion diffusion as a generative prior. In: ICLR (2024)
19. Song, J., Meng, C., Ermon, S.: Denoising diffusion implicit models. In: ICLR (2021)
20. Truebones: Free FBX/.bvh zoo: over 75 animals with animations and textures. https://truebones.gumroad.com/l/skZMC, last accessed: June 2026
21. Tselepi, E., Thermos, S., Potamianos, G.: Controllable single-shot animation blending with temporal conditioning. In: ICCV Workshops (2025). https://doi.org/10.1109/ICCVW69036.2025.00050
22. Wang, X., Ruan, K., Zhang, X., Wang, G.: Animo: Species-aware model for text-driven animal motion generation. In: CVPR (2025). https://doi.org/10.1109/CVPR52734.2025.00186
23. Zhou, Y., Barnes, C., Lu, J., Yang, J., Li, H.: On the continuity of rotation representations in neural networks. In: CVPR (2019). https://doi.org/10.1109/CVPR.2019.00589

Supplementary Material

In this supplementary material, we first provide implementation details (Sec. A), followed by a more in-depth description of the evaluation metrics (Sec. B) used in the paper. Finally, we present additional results (Sec. C) and analysis (Sec. D). The supplementary material also includes qualitative results in video format, accessible by opening `index.html` in a web browser. The webpage is organized according to the three experimental settings discussed in the paper: in-skeleton blending, cross-skeleton blending, and retargeting. Each section presents representative video examples and, when applicable, side-by-side comparisons with Blender, AnyTop, and Motion2Motion.

A Implementation details

The *Semantic Encoder* and *Stochastic Decoder* employ a symmetrical configuration consisting of $L = 4$ blocks with a latent dimensionality of $H = 128$. The `Enrichment Block` projects the $D = 13$ joint-wise features into the H-dimensional space. The diffusion process is set with $T = 100$ timesteps, which are integrated into the stochastic decoder using sinusoidal embeddings. Model training is driven by Eq. 2 with $\lambda_{rot} = 1$. We train for 500k steps on a single NVIDIA RTX 3090 GPU (24 GB VRAM) using the AdamW optimizer. Hyperparameters include a batch size of 10 and an initial learning rate of 1×10^{-4}. We utilize a StepLR scheduler that reduces the learning rate every 10k steps.

We preprocess the Truebones Zoo [20] dataset following the pipeline described in [8]. Each joint for every character is represented as described at Sec. 3.1. To ensure consistency across diverse topologies, we apply character-level Z-score normalization by computing the mean and standard deviation for each specific entity. Skeletons are rotation-normalized to face the $+Z$ direction using the hips and shoulder joints (or their anatomical equivalents) as reference and are grounded by setting the feet at height $Y = 0$. Motion clips are restricted to a maximum length of 200 frames; sequences exceeding this threshold are subdivided into multiple segments. Training is performed on clips of length 40 sampled randomly at each epoch.

B Evaluation Metrics

Evaluation relies on deep features for FID and root-relative positions for all other metrics. Calculations are performed on the transition zone, which corresponds to the entire target clip in the specific case of motion retargeting.

Table 5: Runtime analysis of the blending pipeline.

Components	Overall ↓	Seconds / Joint ↓	Seconds / Frame ↓
Sampling	2.487 ± 0.148	0.051 ± 0.003	0.022 ± 0.001
DDIM Inversion	4.625 ± 0.289	0.096 ± 0.006	0.041 ± 0.002
IK	13.655 ± 1.483	0.283 ± 0.031	0.121 ± 0.013
Full pipeline	20.767 ± 1.602	0.430 ± 0.033	0.185 ± 0.014

FID (Fréchet Inception Distance). This metric assesses generative fidelity by comparing the synthesized motion distribution against the ground truth. Since no pretrained topology-agnostic motion feature extractor exists, we use the *Semantic Encoder* as feature space and complement FID with joint-space metrics where applicable.

Proximity. Mathematically equivalent to the global diversity score in [13], this metric measures the minimum distance between the synthesized transition and the source animations. In the context of blending, it is interpreted as a "lower-is-better" score where smaller values indicate higher fidelity to the reference and target motions. It serves as a joint-space complement to FID, ensuring the synthesized results remain highly correlated with the control sequences.

Jerk Ratio. This measures motion smoothness by computing the third-order derivative of joint positions with respect to time. The ratio is derived by normalizing the synthesized jerk against the character's average ground truth jerk, where a value of 1.0 represents average dynamics. It provides critical insight into motion artifacts, such as high-frequency flickering or unnatural jitter, that simple positional metrics might overlook.

IntraDivDiff (Intra-Diversity Ground Truth Difference). Adopted from [8], this metric measures the absolute difference between the internal statistical diversity of the generated motion and the ground truth set. A value of 0 indicates that the generated transition possesses internal variability identical to the character's average motion statistics. Higher values suggest a deviation from natural movement, making lower scores preferable for maintaining character realism.

C Additional Results

Runtime Analysis Tab. 5 reports the runtime of the main components of our pipeline. The largest computational cost is due to the inverse kinematics (IK) post-process, which dominates the overall runtime. In comparison, the learned stages are relatively efficient: sampling requires approximately 2.5 seconds, and DDIM inversion approximately 4.6 seconds. The complete pipeline takes about 21 seconds per blend, with per-joint and per-frame costs scaling moderately with the size of the skeleton and the sequence length. While this does not enable real-time interaction, it is well suited to offline animation authoring, where the runtime remains negligible compared to the manual effort typically required to

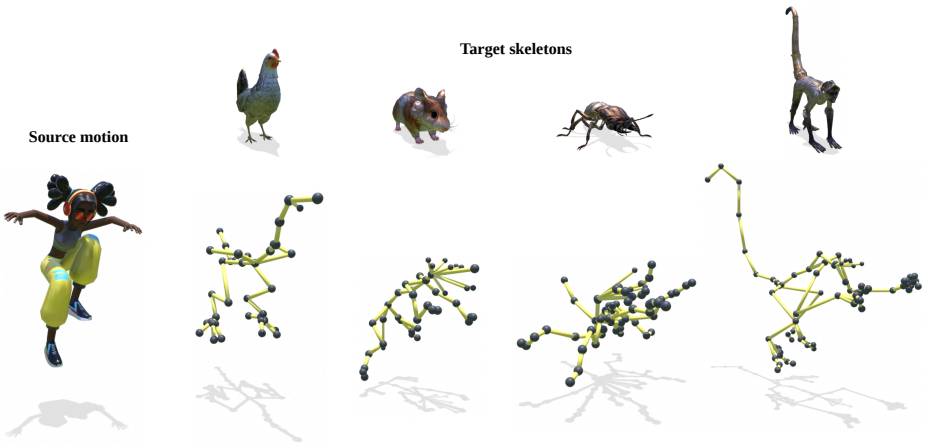

Fig. 8: Zero-Shot Retargeting from Unseen Topologies

an animator to set up correspondences, retarget motions, and clean up a blended transition.

Cross-Dataset Retargeting. Fig. 8 illustrates an additional generalization experiment in which the source motion originates from a humanoid character. Since training is performed exclusively on the Truebones Zoo dataset, the human skeleton is entirely unseen during training. Nevertheless, the model successfully transfers the motion to morphologically diverse target characters, including species with fundamentally different limb structures and joint counts. This result suggests that the learned semantic space captures motion patterns at a level of abstraction that is not tied to specific skeletal configurations encountered during training.

D Latent Space Analysis

To evaluate if the semantic encoder structures its latent space towards a meaningfull topology-agnostic space, we perform a k-nearest-neighbour (k-NN) classification probe. All motions from the Truebones Zoo dataset are encoded, and their per-frame latents are averaged into a single descriptor. We evaluate a k-NN classifier at $k \in \{1, 5, 15, 30\}$ across two downstream tasks: (i) skeleton category (`flying`, `snakes`, `quadrupeds`, `millipeds`, `bipeds`) and (ii) action semantics (`attack`, `idle`, `run`, `die`, `walk`, `other`). These labels are derived manually from filenames, making them highly noisy due to naming inconsistencies and the dataset's nature. Crucially, they are used strictly at evaluation time.

As shown in Figure 9, the encoder implicitly organizes the latent space along morphological and behavioral axes, with all F1-Score probes performing statistically well above the random-chance baseline. This baseline is defined as $\max(1/C, \text{majority-class rate})$, where C is the class count. For skeleton categories (left), the F1-Score at $k \in \{5, 15\}$ reaches ~ 0.80 against a ~ 0.48 base-

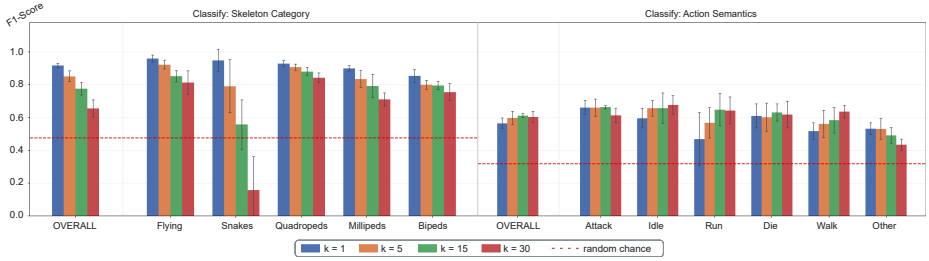

Fig. 9: **k-NN classification of the semantic latent space.** F1 scores at $k \in \{1, 5, 15, 30\}$ for skeleton category (*left*) and action semantics (*right*). The dashed red line indicates the random-chance baseline.

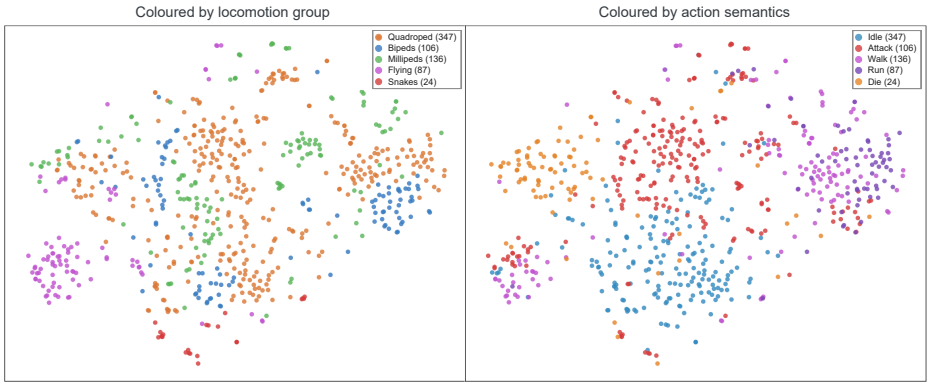

Fig. 10: **t-SNE visualization of the semantic latent space.** The embedding is colored by locomotion group (*left*) and action semantics (*right*), demonstrating local semantic coherence across both dimensions.

line, degrading gracefully except for the underrepresented `snakes` category. For action semantics (right), the F1-Score reaches ~ 0.58 against a ~ 0.31 baseline. This result is highly significant given the severe label noise and high dataset complexity, where each character averages 20–30 motion samples spanning very diverse actions. Consequently, distinct behaviors like `attack` and `idle` are highly reliable, while heterogeneous classes like `run` and `other` show more confusion. This structure emerges naturally because the reconstruction objective forces the encoder to capture both body morphology and behavior.

Figure 10 provides a qualitative t-SNE projection of these descriptors. Colored by locomotion group (left) or action semantics (right), the embedding exhibits emerging local neighborhood without global clustering. This simultaneous encoding ensures that semantically similar motions align across different skeleton types, providing the latent structure necessary to support cross-topology blending.